Digital System Design and Implementation

Final Project

RISC-V RV32 CPU with a Memory-Mapped CNN Coprocessor

Table of contents

A. Introduction 2

B. Design description 2

C. CPU 5

(a.) add 5

(b.) sub 5

(c.) lw 6

(d.) sw 6

(e.) addi 6

(f.) beq (Branch on Equal) 6

(g.) blt (Branch less than) 6

D. Convolutional Neural Network (CNN)[5][6] 8

(a.) What is a Convolutional Neural Network? 8

(b.) Convolution in Action: The Heartbeat of CNN Computation 8

(c.) Data Representation and Numerical Precision Control 9

E. Description 10

F. Grading Rubric (100pts) 11

(a.) Functionality: 60% 11

(b.) Ranking: 30% 12

(c.) Report: 10% 13

G. Submission Requirement 13

H. Reference 13

Due Date: 2026/06/04 (Thur.) 17:59:59

Late submissions are strictly not accepted under any circumstances.

A. Introduction

RISC-V is an open and modular instruction set architecture (ISA) that enables lightweight CPU implementations and flexible customization, making it widely used in embedded systems, IoT devices, and as a host controller in accelerator-based designs. Meanwhile, Convolutional Neural Networks (CNNs) are a cornerstone of modern computer vision, powering tasks such as image classification and object detection. Because CNN inference is dominated by highly regular multiply–accumulate operations and exhibits strong data reuse, it is commonly accelerated by dedicated hardware to improve throughput and energy efficiency.

B. Design description

In this project, you will build a minimal heterogeneous system based on a provided multi-cycle RV32 RISC-V CPU[1] using a Harvard architecture[2], and integrate it with a memory-mapped CNN coprocessor, as shown in **Figure 1.** The data memory is a shared memory with a depth of 1024 and a width of 32 bits, accessed through two ports: Port A is connected to the RV32 CPU and the test circuit, while Port B is connected to the CNN coprocessor.

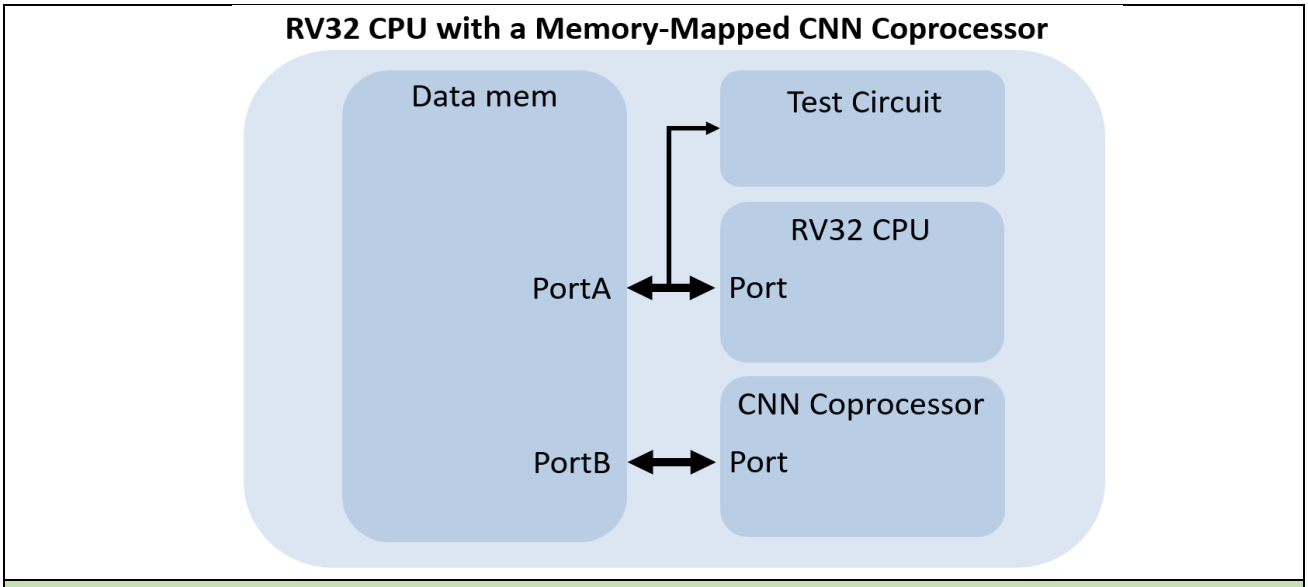

Figure 1 system architecture

The test circuit, which will be provided by us, is responsible for initializing the patterns stored in the shared memory and later reading back the computation results from the shared memory for verification. Except during the initialization and result-checking phases, the test circuit remains passive, effectively acting as a monitor without occupying Port A or interfering with other modules accessing the shared memory.

The RV32 CPU serves as the host processor, configuring and controlling the CNN coprocessor through predefined memory-mapped control/status addresses. After being launched by the CPU, the CNN coprocessor autonomously reads input data and weights from the shared memory and writes the computed output data back to the shared memory.

The overall design flow is shown in **Figure 2**

1. Test circuit initializes Data Memory.
2. Test circuit sets Data Memory[11][0] = 1.
3. CPU detects start bit, clears Data Memory[11][0] to 0.
4. CPU generates Bias0/Bias1 and writes to Data Memory[14]/[15].
5. CPU executes instruction testing
6. Students design their own CPU-CNN coordination mechanism.
7. System writes output feature map start address to Data Memory[13][10:1].
8. System sets Data Memory[13][0] = 1.
9. Test circuit reads result and verifies correctness.

Figure 2 Overall Design Flow

To keep the CPU scope focused, **only a small instruction subset is required: lw, sw, add, addi, sub, beq and blt**. Since the CPU already supports lw, sw, add, and sub, your main tasks on the CPU side are to **complete addi, beq and blt.**

The CNN coprocessor is the main component to be designed by the students. Except for the predefined interfaces, memory map, and required functionalities specified in this project, students are free to determine both the internal architecture of the CNN coprocessor and the system-level coordination mechanism between the CPU and the CNN coprocessor through the shared memory architecture.

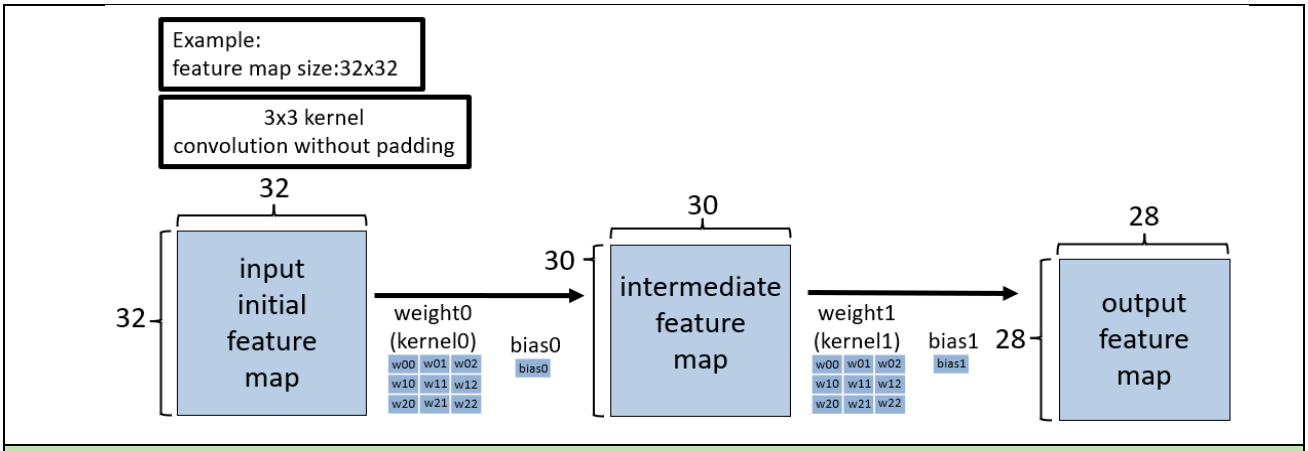

Figure 3 A two-stage convolutional neural network (CNN)

Figure 3 shows the target CNN model executed by the circuit, where the illustrated 32×32 case is only an example. The network topology is fixed, and the only configurable parameter is the feature map size, which ranges from 10 to 32.

The input initial feature map and convolution weights are stored in the shared data memory and can be accessed according to the memory layout shown in Figure 3.

In contrast, the bias values are not directly provided in data memory; they are generated by the CPU after performing the required computation.

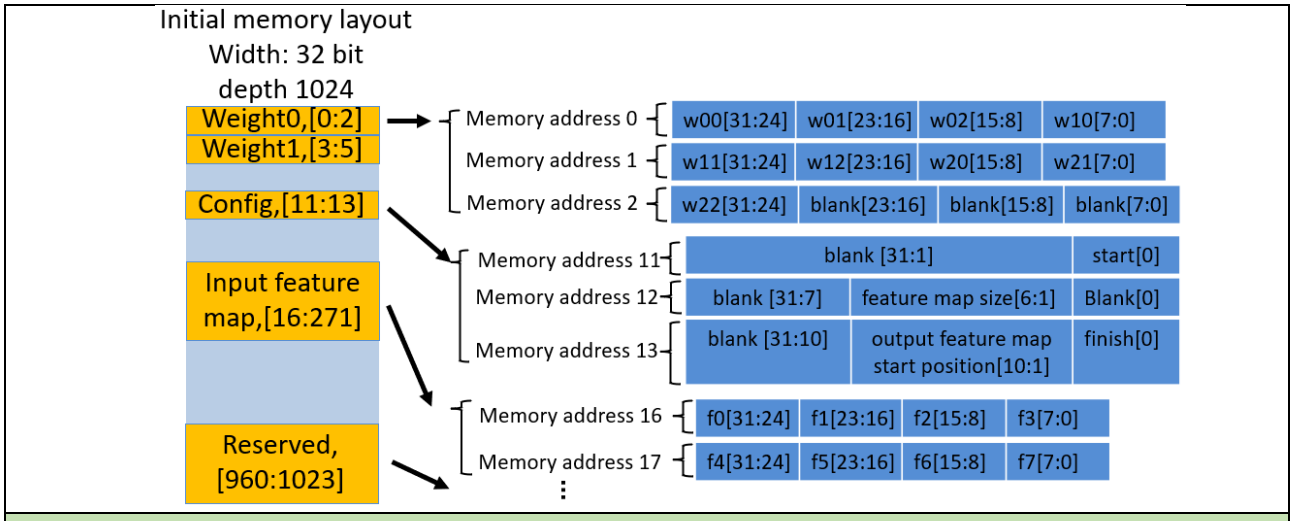

Figure 4 Data memory layout

☠ All orange predefined memory regions are provided by TAs and must be treated as read-only, except for address 11[0](start bit). Students are not allowed to overwrite these locations. If any of these regions are overwritten and the test circuit result is affected, no arguments or appeals will be accepted.

Figure 4 above shows the predefined memory layout of the shared data memory. The memory has a width of 32 bits and a depth of 1024 words. At initialization, memory addresses **0 to 2** are used to store the first 3×3 convolution kernel (Weight0), with each weight represented in 8-bit format and packed into 32-bit memory words. Memory addresses 11 to 13 are allocated for configuration data.

The **start** field at **address 11[0]** is an **active-high control signal** provided by the test circuit. After detecting this start signal and beginning execution, the CPU should clear this bit back to 0. At the time the start signal is asserted, the **feature map size** field at **address 12[6:1]** contains a value in the range from **10 to 32 set by the test circuit**. Regardless of the selected feature map size, the input feature map is always stored beginning at **memory address 16**.

Before computation completes, the system must update **address 13[10:1]** to indicate the starting address of the output feature map in shared memory. The output feature map should be stored using the same layout convention as the input feature map. After the computation is finished, **address 13[0]** must be set high to indicate completion. The remaining upper memory region may be used for student-defined purposes, except for addresses 960 to 1023, which are reserved for grading and must not be overwritten.

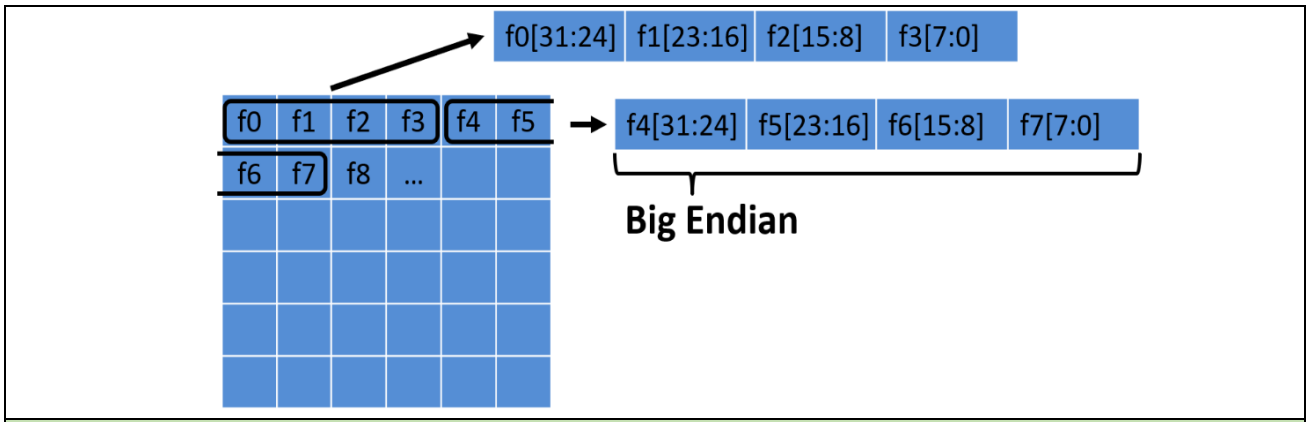

Figure 5 big endian 32-bit word packing of initial feature

Figure 5 above shows the storage layout of the feature map in shared memory. Feature values are stored in row-major order. Each 32-bit word contains four 8-bit feature values packed in big-endian order.

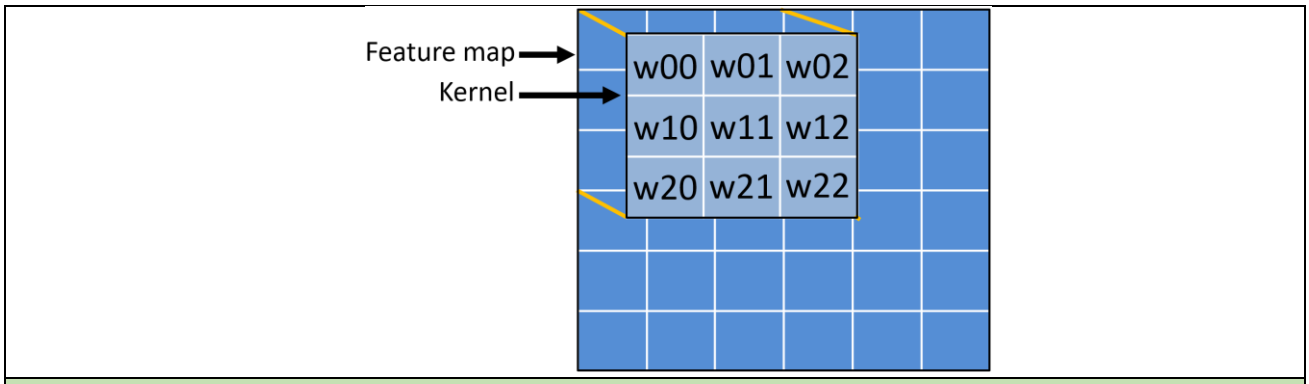

Figure 6 3x3 kernel weight layout (w00~w22 ordering)

Figure 6 shows how each **3×3 kernel weight** maps to the corresponding feature map elements for the convolution **MAC** operation.

C. CPU

The CPU design is based on the multi-cycle architecture. To meet the system requirements for the final project, the baseline CPU has been extended to support additional control-flow instructions and optimized for better performance and area efficiency.

※ Instruction Set Architecture (ISA) Extension:

In addition to the basic instructions (lw, sw, add, sub), three specific instructions—addi, beq and blt—have been implemented to manage the control flow between the host CPU and the CNN coprocessor. The encoding formats for these additional instructions are detailed below:

Additional Instructions:

(a.)add

Definition:

An arithmetic instruction that adds the values of two source registers and writes the result to the destination register.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
0000000	rs2	rs1	000	rd	0110011

Functions:

$$R[rd] = R[rs1] + R[rs2]$$

(b.)sub

Definition:

An arithmetic instruction that subtracts the second source register from the first source register and writes the result to the destination register.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
0100000	rs2	rs1	000	rd	0110011

Functions:

$$R[rd] = R[rs1] - R[rs2]$$

(c.) lw

Definition:

An load instruction that reads a 32-bit word from memory and writes it to the destination register.

[31:20]	[19:15]	[14:12]	[11:7]	[6:0]
imm[11:0]	rs1	010	rd	0000011

Functions:

$$R[rd] = mem[R[rs1] + imm]$$

(d.)sw

Definition:

An store instruction that writes a 32-bit word from a source register to memory.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011

Functions:

$$mem[R[rs1] + imm] = R[rs2]$$

(e.) addi

Definition:

An immediate arithmetic instruction that adds a sign-extended 12-bit immediate value to a source register.

[31:20]	[19:15]	[14:12]	[11:7]	[6:0]
imm[11:0]	rs1	000	rd	0010011

Functions:

$$R[rd] = R[rs1] + imm$$

(f.) beq (Branch on Equal)

Definition:

A conditional branch instruction that performs an equality comparison between two source registers.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
imm[12][10:5]	rs2	rs1	000	imm[4:1][11]	1100011

Functions:

$$if(R[rs1] == R[rs2]), \quad pc = pc + imm;$$
$$else, \quad pc = pc + 4$$

(g.) blt (Branch less than)

Definition:

A conditional branch instruction used for inequality comparison.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
imm[12][10:5]	rs2	rs1	100	imm[4:1][11]	1100011

Functions:

$$if(R[rs1] < R[rs2]), \quad pc = pc + imm;$$
$$else, \quad pc = pc + 4$$

※ Bias Generation Specification (Implemented by CPU):

Before the CNN computation starts, the CPU must read the specified input feature map values from Data Memory and generate two bias values: Bias0 and Bias1.The CPU shall generate the bias values as **Figure 7**:

✧ Notice: You can put these

```
addi x1, x0, 16   # x1 = x0(0) + 16
lw   x2, 0(x1)    # x2 = Input feature map[16]
lw   x3, 4(x1)    # x3 = Input feature map[17]
lw   x4, 8(x1)    # x4 = Input feature map[18]
add  x5, x2, x3   # x5 = Input feature map[16] + Input feature map[17]
add  x5, x5, x4   # x5(Bias_0) = Input feature map[16] + Input feature map[17] + Input feature map[18]
sw   x5, 56(x0)   # 把Bias_0放到 Memory Address 14
blt  x3, x2, T    # if Input feature map[17] < Input feature map[16], jump to sub
add  x3, x2, x1
T:
sub  x4, x2, x3   # x4 = Input feature map[16] - Input feature map[17]
addi x4, x4, 1    # x4 = bias_1
sw   x4, 60(x0)   # 把Bias_1放到 Memory Address 15
```

Figure 7 Instruction sequence for bias generation

✧ **Bias Storage Format**

Both **Bias0** and **Bias1** shall be written back to Data Memory [14] and Data Memory [15] as 32-bit words. When the CNN module uses these bias values, it shall extract bits [7:0] from the stored value as the actual bias used for CNN computation.

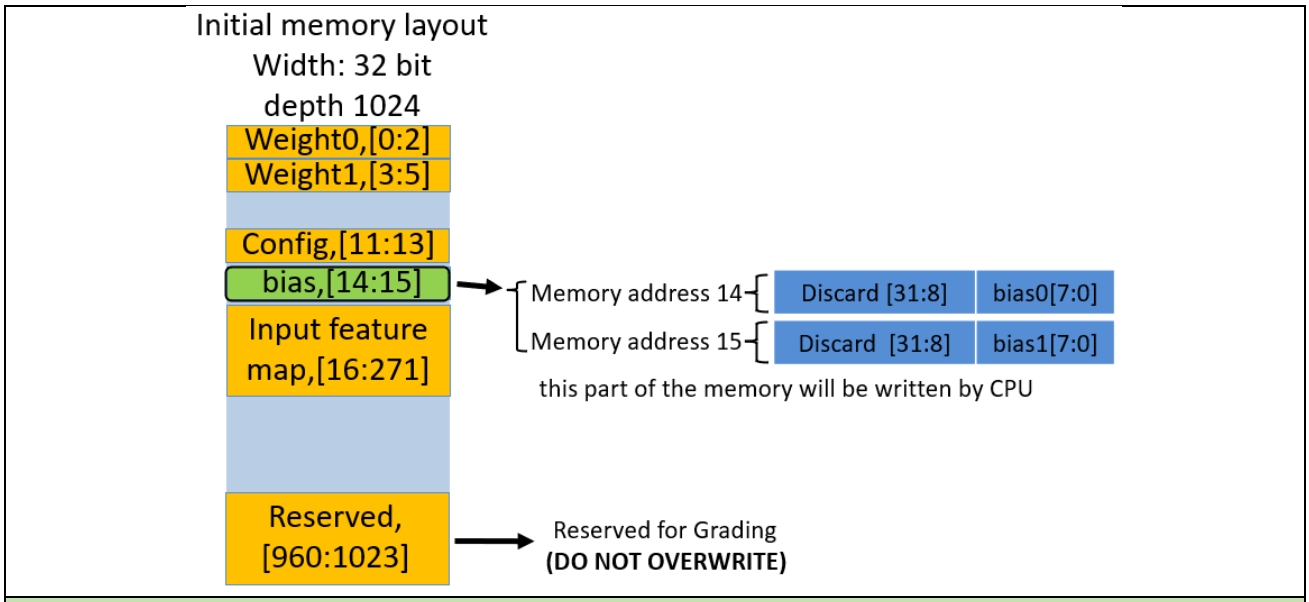

Figure 8 Data memory address mapping for bias storage

※ Instruction Memory Layout:

The instructions in the Instruction Memory are stored in a **contiguous address space**, where each instruction occupies one 32-bit word and is placed in consecutive memory locations. However, the **PC** used by the CPU must still follow the **PC + 4** update rule. In other words, the PC advances according to **byte addressing**, rather than directly incrementing the memory index by 1. Therefore, students must design or derive an appropriate address mapping method so that, even when the PC increases by 4 each time, the processor can still correctly fetch the consecutively stored instructions from the Instruction Memory. Students may arrange the address space of the Instruction Memory based on their own design. However, at least **150 instruction addresses** must be reserved for the provided test instructions.

D. Convolutional Neural Network (CNN)[5][6]

(a.) What is a Convolutional Neural Network?

A Convolutional Neural Network (CNN) is a type of deep learning model inspired by the human visual system. It automatically learns to extract important features from images, such as edges, shapes, and objects, and is widely used in image classification, face recognition, and object detection.

※ CNN computation mainly consists of the following operations:

- Convolution: Uses small filters (kernels) sliding over the image to extract spatial features like edges or textures.
- Activation Function: Adds non-linearity to the model. A common example is ReLU (Rectified Linear Unit), which sets negative values to 0.
- Pooling: Reduces the size of feature maps and the overall computation cost. Common methods include Max Pooling and Average Pooling.
- Fully Connected Layer: Integrates the extracted features for the final classification or prediction.

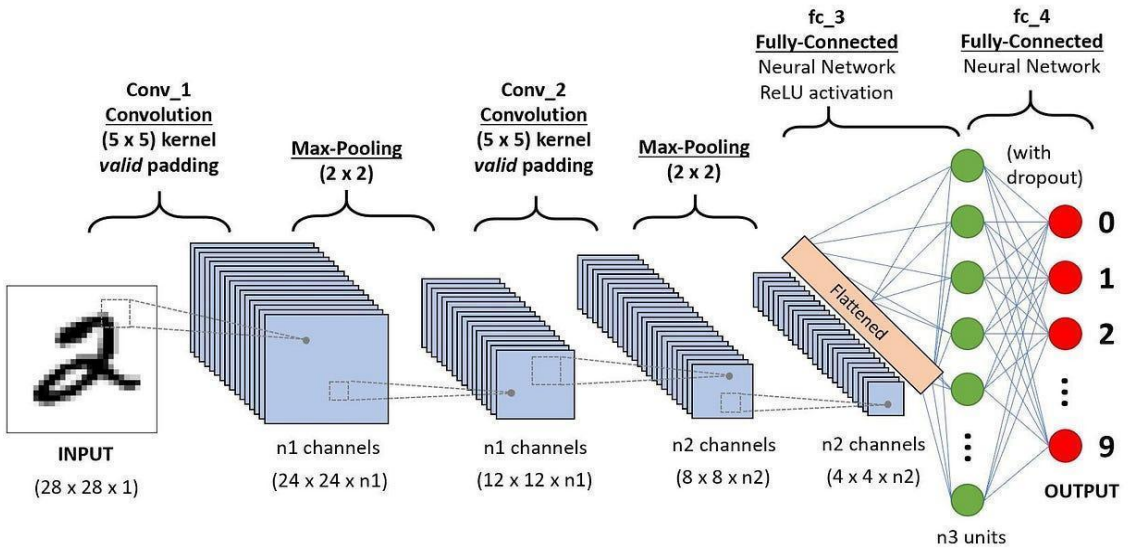

Figure 9 Typical CNN Architecture [5]

※ Overall, CNN computation can be viewed as a series of data transformations:

The input images will undergo feature extraction through convolution, be subsequently fed into fully connected layers for classification, and ultimately yield the prediction. In this project, we will only be implementing Convolution Layers (without activation function).

(b.) Convolution in Action: The Heartbeat of CNN Computation

The Convolutional Layer performs multiply-and-accumulate (MAC) operations on input images or feature maps to extract meaningful features. Its core components include:

- Kernel / weight: A small matrix (e.g., 3×3) used to detect local patterns.
- Stride: The step size when sliding the kernel. (Stride = 1 in the project)
- Padding: The border-handling method that controls output size(Padding = 0 in the project)

$$output(i,j) = \sum (input * weight) + bias$$

In essence, the convolution layer functions as a feature detector. The kernel slides over the input image, and at each position, it computes the dot product between the kernel and the corresponding image region.

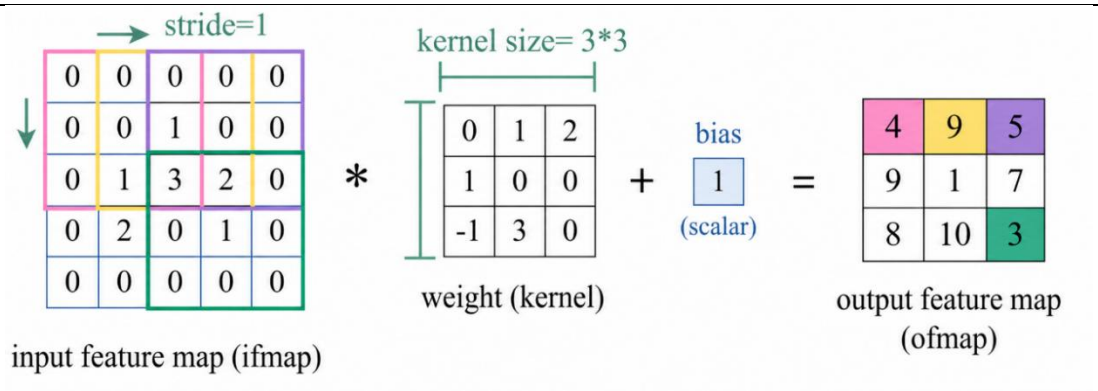

Figure 10 Convolution Operations

(c.) Data Representation and Numerical Precision Control

In CNN hardware implementations, data representation directly affects memory usage, computation cost, and accuracy. In this final project, we will adopt signed Q1.6 data format that reduces storage and hardware complexity, but may introduce rounding errors, overflow, and precision loss. Therefore, precision control methods such as rounding and saturation are needed to keep the computation stable and reliable.

i. Fixed point value

In this project, all input features, weights, and output features are represented in **signed 8-bit fixed-point Q1.6 format**. In this representation, each value is stored in **8-bit two's complement form**, with **1 sign bit, 1 integer bit, and 6 fractional bits**.

$$value = \left(-b_7 * 2^7 + \sum_{i=0}^{6} b_i * 2^i\right) * 2^{-6}$$

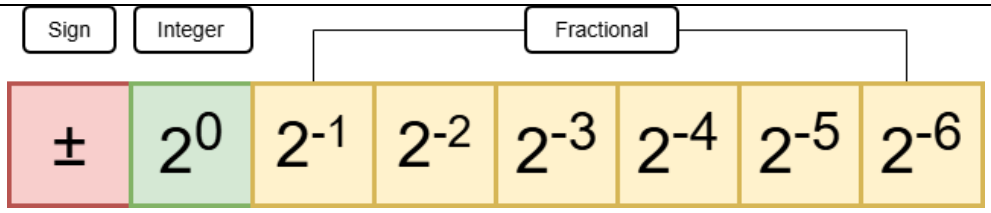

Figure 11 Structure and Value Comprehension of 8-bits signed Q1.6 format

ii. Rounding

In CNNs, operations such as convolution, accumulation, and bias addition often produce intermediate results with higher precision than the target storage format. Since this project stores the final result in signed Q1.6 format, the high-precision value must be rounded to one of the representable signed Q1.6 values after each convolution computation.

1) Round-to-Nearest Rule:
   For a high-precision value x, let the two nearest Q1.6 representable values be q_low and q_high. If x is closer to q_low, round to q_low. If x is closer to q_high, round to q_high.
   e.g. 1(signed Q1.8 to signed Q1.6): 00.01001001 −> 00.010010
   e.g. 2(signed Q1.8 to signed Q1.6): 00.01001011 −> 00.010011
2) Ties-to-Even Rule:
   If x is exactly halfway between q_low and q_high, choose the value whose scaled integer code is even. For signed Q1.6, each value can be scaled as an integer code by multiplying it by 2^6. This avoids always rounding halfway cases upward, which would introduce a positive bias. With ties-to-even rounding, the rounding error is unbiased in expectation, so the expected value of the rounded result is preserved.

iii. Saturation

In quantized CNNs using signed Q1.6, saturation clamping restricts values to the [-2.0, 1.984375] range by mapping any overflow to the nearest representable boundary.

E. Description

The following design requirements must be followed.

1. The name of the top module is RISCV_CNN
2. Saturation follows saturation clamp, which clips the value to the closest representable value.
3. Rounding follows the round-to-nearest, ties-to-even rule.
4. The CNN model used in this project includes bias terms in the convolution operations, meaning that each convolution layer applies both weights and bias during computation.
5. All signed numbers are represented in two's complement format, with the most significant bit (MSB) as the sign bit.
6. Intermediate values during computation should preserve full precision, and only be saturated and rounded to signed Q1.6 format after each convolution computation (refer to figure 10).
7. A Clock Wizard may be instantiated to generate the desired clock frequency.

➢ **Input**

Name	Width	Notes
clk	1	Clock signal that drives sequential circuits. The frequency can be modified. This signal is connected to the pinout P17 on the FPGA.
rstn	1	Asynchronous active low reset signal. This signal is connected to the pinout P15 on the FPGA.
tc	4	tc:4'd0 => CNN testcase0 tc:4'd1 => CNN testcase1 …… tc:4'd9 => CNN testcase9

		tc:4'd10 => CPU testcase0 tc:4'd11 => CPU testcase1 tc:4'd12 => CPU testcase2 tc:4'd13 => CPU testcase3 tc:4'd14 => CPU testcase4 tc:4'd15 => run all testcase tc[3:0] is connected to the pinout:{R2, M4, N4, R1} on the FPGA.
mode	1	mode:1 => view testcase result (use "tc" to control which testcase) mode:0 => run test case mode (use "start" to start testing testcases) (use "tc" to control which testcase) This signal is connected to the pinout P5 on the FPGA.
st	1	Indicating start testing, positive edge triggering. This signal is connected to the pinout R15.

➢ **Output**

Name	Width	Notes
seven_seg	7	Use by test circuit to output processing time and functionality correctness. seven_seg[6] is connected to B4 on the FPGA seven_seg[5] is connected to A4 on the FPGA seven_seg[4] is connected to A3 on the FPGA seven_seg[3] is connected to B1 on the FPGA seven_seg[2] is connected to A1 on the FPGA seven_seg[1] is connected to B3 on the FPGA seven_seg[0] is connected to B2 on the FPGA
anode	4	Use by test circuit to output processing time and functionality correctness. anode[0] is connected to H1 on the FPGA anode[1] is connected to C1 on the FPGA anode[2] is connected to C2 on the FPGA anode[3] is connected to G2 on the FPGA

F. Grading Rubric (100pts)

This final project will be graded based on three components:

Students may not use additional BRAMs in their own design. Only the Data Memory and Instruction Memory are allowed to be BRAMs. The TA's test circuit is for verification only and is not included in the student design resource constraint. Modifying the test circuit or directly outputting the expected answers without real computation is strictly prohibited.

(a.) Functionality: 60%

i. Behavioral simulation-20%

- CPU instruction correctness 10%
  - addi, sw 2%
  - lw 2%
  - sub add 2%
  - beq 2%
  - blt 2%
- CNN computation correctness 10%

ii. Post-synthesis simulation-10%
- CPU instruction correctness 5%
  - addi, sw 1%
  - lw 1%
  - sub add 1%
  - bet 1%
  - blt 1%
- CNN computation correctness 5%

iii. Post-implementation simulation-10%
- CPU instruction correctness 5%
  - Same distribution as Post-synthesis simulation
- CNN computation correctness 5%

iv. On board-20%
- CPU instruction correctness 10%
  - addi, sw 2%
  - lw 2%
  - sub add 2%
  - beq 2%
  - blt 2%
- CNN computation correctness 10%

(b.) Ranking: 30%

This part of the grade is based on ranking the implementation level and result of your AT product.

The AT product is defined as **Area × Processing Time**.

$$Area = (\#Slice\ LUTs + \#Slice\ Registers + \#F7\ Muxes + \#F8\ Muxes + 280 * \#DSPs)$$

$$Processing\ time = Total\ processing\ time\ of\ the\ testcases\ in\ micro\ seconds$$

Projects will be classified into four levels: **A, B, C, and D**, according to the completeness of the design flow and verification results.

Level	Criteria
A	The design is fully completed and passes RTL simulation, synthesis, implementation, and post-implementation timing simulation. Projects in Level A will be ranked by **AT (Area × Time)**. A smaller AT value indicates a better design.
B	The design passes RTL simulation, synthesis, and implementation, but post-implementation timing simulation is not completed or not passed. Projects in Level B will be ranked first by the **simulation score**, and then by **AT** if the simulation scores are the same.
C	The design passes RTL simulation and synthesis, but implementation is not completed or not successful. Projects in Level C will be ranked first by the simulation score, and then by **AT** if the simulation scores are the same.
D	The design only completes RTL-level simulation, or some required functions are incomplete. Projects in Level D will be ranked first by the **simulation score**.

For **Levels B, C, and D**, the simulation score has higher priority than AT, since functional correctness is considered more important than optimization for designs that have not completed the full verification flow.

(c.) Report: 10%

i. Architecture block diagram
ii. Design technique description
iii. STA summary
iv. Hardware Utilization
v. Behavior simulation result (screenshot simulation result)
vi. Post-synthesis simulation result (screenshot simulation result)
vii. Post-implementation simulation result (screenshot simulation result)

※ Bonus

(a.) Pipelined CPU design **(5pts)**
(b.) Special design techniques sharing (?pts)

G. Submission Requirement

You need to submit three files. Please upload these files to ee-class.

1. The source code.
   Please name it as "studID_name_FP.v" (ex:114501000_陳聿廣_FP.v) .If there were multiple dependent verilog code files, please merge to one file and name it as mentioned above.
2. The bitstream file.
   Please name it as "FP_studID.bit" (ex: FP_114501000.bit)
3. A report named studID_FP_report.pdf (ex: 114501500_FP_report.pdf).

Note that the only acceptable report file format is .pdf, no .doc/.docx or other files are acceptable. BE SURE to follow the naming rule mentioned above. Otherwise, your program will not be graded.
We don't restrict the report format and length. In your report, you have to at least describe:

- The degree of completion of the assignment; (If you complete all requirements, just specify all)
- Briefly explain how you completed the assignment. Your workflow or only a simple description of your logic are all accepted.
- The hardness of this assignment and how you overcome it
- Any suggestions about this assignment?

Demo session will be held. Please submit your assignment on time. If you have questions, please E-mail to both professor (andyygchen@ee.ncu.edu.tw, yhhuang@ncu.edu.tw) and TA 沈侯汰 (houtaishen@gmail.com).

H. Reference

[1] Multi-Cycle Architecture:
https://enesharman.medium.com/single-cycle-vs-multi-cycle-processors-1c5bf468c569

[2] Harvard architecture
https://en.wikipedia.org/wiki/Harvard_architecture
[3] CPU polling
https://www.totalphase.com/blog/2023/10/polling-interrupts-exploring-differences-applications/?srsltid=AfmBOor3PyQ_6oTYzwSoo_4cGwOvDZxoGmduwe_Wy7ZJ0YHrVENMhXux
[4] Pipelined Architecture:
https://www.geeksforgeeks.org/computer-organization-architecture/pipelined-architecture-with-its-diagram/
[5] CNN
https://medium.com/@alejandro.itoaramendia/convolutional-neural-networks-cnns-a-complete-guide-a803534a1930
[6] CNN
https://www.youtube.com/watch?v=OP5HcXJg2Aw&list=PLJV_el3uVTsMhtt7_Y6sgTHGHp1Vb2P2J&index=9

Some of the reference designs below may be useful for your implementation:

[7] Systolic array
https://zhuanlan.zhihu.com/p/605590757
[8] Handshake
https://www.cnblogs.com/mikewolf2002/p/11345401.html